

AI 협업 도구 도입

활동별 워크플로우 정의 문서

대상: AI 도구를 처음 쓰는 1 티어 직원

범위: Claude · Figma MCP · Claude Code · Obsidian · GitHub

작성일: 2026-06-09

목차

이 문서를 읽기 전에

이 문서는 AI 도구를 한 번도 써본 적 없는 분이 그대로 따라 할 수 있도록 만들었습니다. 전문 용어는 처음 나올 때 풀어서 설명합니다. 순서대로 읽고, 각 활동의 '단계별 활동 정의'를 그대로 따라 하면 됩니다.

핵심 용어 빠르게 익히기

AI / LLM: 사람이 쓴 글을 이해하고 답을 만들어 주는 프로그램. 이 문서에서는 Claude 가 그 역할을 합니다.

프롬프트(Prompt): AI 에게 시키는 '지시문'. 사람에게 일을 부탁하듯 글로 적으면 됩니다.

터미널(Terminal): 글자만으로 컴퓨터에 명령을 내리는 검은 창. Mac 은 '터미널', Windows 는 'PowerShell'입니다.

MCP: AI(Claude)가 다른 프로그램(예: Figma)을 직접 다룰 수 있게 연결해 주는 '다리'라고 생각하면 됩니다.

리포지토리(Repository, 줄여서 repo): GitHub 에 파일을 보관하는 '폴더'. 변경 이력이 자동으로 남습니다.

Vault(볼트): Obsidian 에서 메모를 모아 두는 '노트 폴더'입니다.

전체 업무 흐름 한눈에 보기

아래 세 가지 활동을 순서대로 진행합니다. 1 번을 끝내야 2 번, 3 번이 수월해집니다.

순서	활동	한 줄 목표	예상 소요
1	도구 세팅	Claude·Figma MCP·Claude Code·Obsidian·GitHub 를 설치하고 서로 연결	2~3 시간
2	이전 작업 캐치업	WBS 등 기존 산출물을 읽고 현재 상태를 파악	1~2 시간
3	앱 만들기	문서 초안 생성기(웹앱)를 직접 제작	1~2 일

활동 1. 도구 세팅

1.1 요약

이 활동의 목표는 다섯 가지 도구를 한 대의 컴퓨터에 설치하고 서로 연결하는 것입니다. 연결이 끝나면 Claude 가 디자인 파일(Figma)을 읽고, 코드를 작성(Claude Code)하고, 메모(Obsidian)와 코드 저장소(GitHub)가 자동으로 백업되는 환경이 갖춰집니다.

이 활동을 마치면 할 수 있는 것

Claude 앱에서 대화로 작업 지시하기
Figma 디자인을 Claude 가 읽어 코드로 변환하기
터미널에서 Claude Code 로 파일을 직접 만들고 수정하기
Obsidian 메모가 GitHub 에 자동 백업되기

1.2 워크플로우

아래 순서를 그대로 따릅니다. 앞 단계가 끝나야 다음 단계가 동작합니다.

1. 준비물 확인 — 계정과 Node.js
2. Claude 앱(데스크톱) 설치 및 로그인
3. Claude Code 설치 (터미널 도구)
4. GitHub 계정·리포지토리 준비
5. Obsidian 설치 후 Git 연동으로 GitHub 자동 백업
6. Figma MCP 를 Claude Code 에 연결
7. 전체 연결 점검

1.3 단계별 활동 정의

단계 1 — 준비물 확인

Claude Code 는 Node.js 라는 프로그램 위에서 동작합니다. Node.js 는 18 버전 이상이 필요하며, 안정성을 위해 20 LTS 버전이 권장됩니다.

확인 방법: 터미널을 열고 아래를 입력합니다.

할 일	입력/확인 내용
Node.js 버전 확인	node --version 입력 → v18.0.0 이상이면 OK
버전이 낮거나 없으면	nodejs.org 에서 LTS 버전을 내려받아 설치
계정 준비	Claude 유료 구독(또는 API 키), GitHub 계정, Figma 계정

npm 설치 시 sudo(관리자 권한)를 쓰지 마세요. 권한 오류가 나면 sudo 대신 nvm(노드 버전 관리자)을 사용하는 것이 올바른 해결책입니다.

단계 2 — Claude 앱 설치

8. claude.ai/download 에서 데스크톱 앱을 내려받아 설치합니다.

9. Anthropic 계정으로 로그인합니다.
10. 간단한 질문을 하나 보내 응답이 오는지 확인합니다.

단계 3 — Claude Code 설치

Claude Code 는 터미널에서 Claude 에게 코딩을 맡기는 도구입니다. 설치 방법은 두 가지입니다.

방법	명령/설명	추천 대상
네이티브 설치(권장)	의존성 없이 자동 업데이트됨. 공식 안내의 설치 명령 실행	대부분의 사용자
npm 설치	<code>npm install -g @anthropic-ai/claude-code</code>	버전 고정이 필요한 경우

11. 설치 후 `claude --version` 으로 설치 확인
12. `claude auth login` 으로 로그인 (브라우저가 열리면 계정 승인)
13. 작업 폴더로 이동해 claude 입력 → 대화 시작

단계 4 — GitHub 준비

14. github.com 에서 계정을 만듭니다(있으면 생략).
15. 우측 상단 + → New repository 로 새 리포지토리를 만듭니다. 메모를 남에게 보이지 않으려면 Private(비공개)로 설정합니다.
16. 나중에 인증에 쓸 Personal Access Token(개인 접근 토큰)을 발급해 둡니다. 토큰은 비밀번호처럼 취급하고 절대 외부에 공유하거나 코드에 적어 올리지 마세요.

⚠ 토큰, 비밀번호 같은 자격 증명은 본인이 직접 입력해야 합니다. 화면이나 문서에 그대로 적어 두지 마세요.

단계 5 — Obsidian + Git 연동

Obsidian 은 메모 앱이고, 'Obsidian Git' 커뮤니티 플러그인을 쓰면 메모가 GitHub 에 자동으로 커밋·푸시(백업)됩니다.

17. Obsidian 을 설치하고 Vault(노트 폴더)를 하나 만듭니다.
18. Settings → Community plugins → Browse 에서 'Git'을 검색해 Obsidian Git 플러그인을 설치·활성화합니다.
19. Vault 폴더를 GitHub 리포지토리와 연결합니다(원격 주소 등록).
20. .gitignore 에 .obsidian/workspace.json 등 기기별 설정 파일을 추가해 불필요한 동기화를 막습니다.
21. `commit-and-sync` 명령으로 백업이 동작하는지 확인합니다.

참고: 모바일에서는 동기화가 느리거나 불안정할 수 있습니다. 데스크톱 사용을 권장합니다.

단계 6 — Figma MCP 연결

Figma MCP 를 연결하면 Claude Code 가 Figma 디자인(컴포넌트·변수·레이아웃)을 직접 읽고 코드로 변환할 수 있습니다. 가장 권장되는 방법은 공식 플러그인 설치입니다.

선택지	설명
원격(Remote) 서버 — 권장	대부분의 사용자에게 맞는 방식이며 기능 폭이 가장 넓습니다. Figma 데스크톱 앱 설치 없이 Figma 파일에 직접 연결됩니다.
데스크톱/Desktop) 서버	특정 조직·엔터프라이즈 용도. Figma 데스크톱 앱이 필요합니다.

22. 터미널에서 `claude plugin install figma@claude-plugins-official` 을 실행합니다.

23. Claude Code 를 재시작한 뒤 `/plugin` 을 입력해 Installed 탭에서 figma 서버를 선택합니다.

24. 열리는 인증 페이지에서 Allow access 를 눌러 Figma 계정 접근을 승인합니다.

25. `/mcp` 명령으로 figma 가 'Connected'로 표시되는지 확인합니다.

 OAuth 접근 권한 승인은 본인이 직접 확인 후 진행하세요. 권한 화면의 요청 범위를 읽고 승인합니다.

단계 7 — 전체 연결 점검

점검 항목	확인 방법	정상 상태
Claude Code	<code>claude --version</code>	버전 번호 표시
GitHub 백업	Obsidian 에서 메모 수정 후 sync	GitHub 에 커밋 반영
Figma MCP	<code>/mcp</code>	figma = Connected

1.4 각 AI 채널 정의

'채널'은 Claude 를 사용하는 창구(인터페이스)를 말합니다. 활동·상황에 따라 알맞은 채널을 고릅니다.

채널	무엇인가	언제 쓰나	입력 방식
Claude 앱(웹/데스크톱)	대화형 채팅 화면	아이디어 정리, 문서 초안, 질문	채팅창에 글
Claude Code	터미널 기반 코딩 에이전트	파일 생성·수정, 코드 작성, 명령 실행	터미널 명령 + 자연어
Figma MCP (채널 아님, 연결)	Claude Code 가 Figma 를 읽는 통로	디자인을 코드로 변환할 때	Claude Code 안에서 사용

정리: 사람과 대화하듯 일을 시키려면 Claude 앱, 실제 파일·코드를 만들게 하려면 Claude Code 를 씁니다. Figma MCP·GitHub·Obsidian 은 그 자체가 AI 는 아니고, Claude 가 다룰 수 있도록 연결되는 '도구'입니다.

활동 2. 이전 작업 캐치업

2.1 요약

앞 사람이 만들어 둔 산출물(WBS 등)을 읽고 '지금 어디까지 진행됐는지'를 파악하는 단계입니다. AI 를 도구로 활용하면 긴 문서를 빠르게 요약하고, 모르는 용어를 바로 물어보며 이해를 높일 수 있습니다.

WBS 란?

WBS(Work Breakdown Structure, 작업 분해 구조): 큰 프로젝트를 작은 작업 단위로 쪼개 트리(나무) 형태로 정리한 표입니다.

'무슨 일을, 누가, 언제까지' 하는지를 한눈에 보여 줍니다.

2.2 워크플로우

26. 기존 산출물 수집 — WBS, 회의록, 디자인 파일 등을 한곳에 모읍니다.
27. Claude 앱에 올려 핵심 요약을 받습니다.
28. 이해되지 않는 부분을 추가 질문으로 명확히 합니다.
29. 파악한 내용을 Obsidian 메모로 정리합니다(자동으로 GitHub 백업).
30. 현재 상태 / 남은 일 / 막힌 부분을 한 페이지로 요약합니다.

2.3 단계별 활동 정의

단계 1 — 산출물 수집

- 문서 파일(WBS 스프레드시트, 회의록 등)과 Figma 디자인 링크를 모읍니다.
- 파일 형식을 확인합니다(엑셀, 워드, PDF 등). Claude 앱은 PDF·이미지·텍스트를 직접 읽을 수 있습니다.

단계 2 — AI 요약 받기

Claude 앱에 파일을 올리고 아래와 같이 지시합니다.

프롬프트 예시 (그대로 복사해서 쓰세요)

이 WBS 를 처음 보는 사람이 이해할 수 있게 요약해 줘. (1) 프로젝트 목표 (2) 이미 끝난 작업 (3) 진행 중인 작업 (4) 아직 시작 안 한 작업 순서로 정리하고, 전문 용어가 있으면 풀어서 설명해 줘.

단계 3 — 추가 질문으로 명확화

- "이 항목이 무슨 뜻인지 더 쉽게 설명해 줘" 처럼 모르는 부분을 바로 물어봅니다.
- "이 작업이 끝나려면 무엇이 더 필요해?" 같이 빠진 정보를 확인합니다.

단계 4 — Obsidian 에 정리

- 파악한 내용을 메모로 적어 두면 활동 1 에서 설정한 Git 연동으로 GitHub 에 자동 백업됩니다.

단계 5 — 현재 상태 한 장 요약

- '현재 상태 / 남은 일 / 막힌 부분(blocker)' 세 칸으로 정리합니다.

2.4 각 AI 채널 정의

채널	이 활동에서의 역할	주의점
Claude 앱	문서 요약, 용어 설명, 질의응답	민감 정보는 올리기 전 사내 규정 확인
Obsidian	파악한 내용 기록·백업	메모가 자동 백업되는지 확인

이 단계에서는 Claude Code 나 Figma MCP 는 필수가 아닙니다. 디자인 맥락이 필요할 때만 Figma 링크를 Claude 에 함께 제공하면 됩니다.

활동 3. 앱 만들기 — 문서 초안 생성기

3.1 요약

문서 유형(회의록·WBS·IA 등)을 고르고 핵심 내용만 입력하면, 그 형식에 맞는 초안을 자동으로 만들어 주는 웹앱을 제작합니다. Claude Code 가 코드를 작성하고, 필요하면 Figma 디자인을 참고하며, 결과물은 GitHub 에 보관합니다.

IA 란?

IA(Information Architecture, 정보 구조): 화면·메뉴·콘텐츠를 어떻게 배치하고 연결할지 설계한 '구조도'입니다.

쉽게 말해 '어떤 정보가 어디에 들어가는가'를 정리한 지도입니다.

3.2 워크플로우

31. 요구사항 정의 — 어떤 문서 유형을, 어떤 입력으로, 어떤 형식의 초안으로 만들지 정합니다.
32. (선택) Figma 에서 화면 디자인을 잡고 Figma MCP 로 Claude Code 에 전달합니다.
33. Claude Code 에 앱 제작을 지시해 코드를 생성합니다.
34. 브라우저에서 직접 실행해 동작을 확인합니다.
35. 문서 유형별 형식(템플릿)과 프롬프트를 다듬습니다.
36. GitHub 에 코드를 저장(커밋·푸시)합니다.

3.3 단계별 활동 정의

단계 1 — 요구사항 정의

항목	정해야 할 내용	예시
지원 문서 유형	앱에서 고를 수 있는 종류	회의록, WBS, IA
입력 항목	사용자가 넣는 핵심 정보	회의록 → 참석자·안건·결정사항
출력 형식	초안의 구조	제목 → 요약 → 항목별 정리

단계 2 — (선택) Figma 디자인 연결

- Figma 에서 화면을 디자인했다면, 프레임을 선택하거나 링크를 복사해 Claude Code 에 전달합니다.
- Claude Code 는 디자인의 색상·간격·레이아웃을 읽어 그에 맞는 코드를 생성합니다.
- 디자인이 없다면 이 단계는 건너뛰고, Claude Code 에 글로 화면을 설명해도 됩니다.

단계 3 — Claude Code 로 앱 생성

터미널에서 앱을 만들 폴더로 이동한 뒤 claude 를 실행하고 아래처럼 지시합니다.

프롬프트 예시

문서 초안 생성기 웹앱을 만들어 줘. 사용자가 문서 유형(회의록/WBS/IA)을 드롭다운에서 고르고,

핵심 내용을 입력하면, 그 유형 형식에 맞는 초안을 만들어 화면에 보여 주는 한 페이지짜리 앱이야. HTML·CSS·JavaScript로 만들고, 결과는 복사 버튼으로 가져갈 수 있게 해 줘.

초안을 AI가 직접 생성하게 하려면, 앱 안에서 AI 응답을 받아 처리하도록 Claude Code에 추가로 요청할 수 있습니다.

단계 4 — 실행·확인

- Claude Code가 만든 파일을 브라우저에서 열어 동작을 확인합니다.
- 문제가 있으면 "이 부분이 이렇게 동작하는데, 이렇게 바꿔 줘"라고 다시 지시합니다.

단계 5 — 템플릿·프롬프트 다듬기

- 문서 유형마다 출력 형식이 적절한지 점검하고, 표현을 조정합니다.
- 자주 쓰는 입력 예시를 넣어 두면 사용자가 쓰기 쉬워집니다.

단계 6 — GitHub에 저장

- 완성된 코드를 GitHub 리포지토리에 커밋·푸시해 보관·공유합니다.
- Claude Code에 "이 변경사항을 커밋하고 푸시해 줘"라고 지시하면 git 작업도 대신 처리합니다.

3.4 각 AI 채널 정의

채널	이 활동에서의 역할	입력 예
Claude 앱	요구사항 정리, 문서 형식 설계, 프롬프트 문구 다듬기	"회의록 초안 형식을 설계해 줘"
Claude Code	앱 코드 생성·수정, 실행, git 커밋	"드롭다운을 추가해 줘"
Figma MCP	디자인을 읽어 코드로 반영(선택)	Figma 프레임 선택 후 변환 요청
GitHub	코드 보관·이력 관리·공유	커밋·푸시

공통 주의사항 (모든 활동)

- 자격 증명 직접 입력: 비밀번호·토큰·API 키는 본인이 직접 입력하고, 문서나 코드에 그대로 적어 올리지 않습니다.
- 민감 정보 점검: 외부에 올리는 파일에 고객 정보·내부 기밀이 없는지 확인합니다.
- 권한 승인 확인: OAuth·접근 권한 화면은 요청 범위를 읽고 승인합니다.
- 결과 검토: AI가 만든 초안·코드는 그대로 쓰지 말고 사람이 한 번 검토합니다.
- 막히면 질문: 오류 메시지를 그대로 Claude에 붙여 넣고 해결 방법을 물어봅니다.

외부 참조 출처

아래는 각 도구의 공식 또는 신뢰할 수 있는 문서입니다. 설치·연결 중 막히면 해당 출처를 우선 확인하세요. (확인일: 2026-06-09)

Claude Code

- [Claude Code 설치·시스템 요구사항 \(공식 문서\)](#)

Figma MCP

- [Figma MCP 서버 설정 — Claude Code \(Figma 공식 헬프센터\)](#)
- [원격\(Remote\) Figma MCP 서버 설정 안내 \(Figma 헬프센터\)](#)
- [Remote 서버 설치 \(Figma Developer Docs\)](#)
- [Figma MCP 서버 가이드 \(GitHub\)](#)

Obsidian + Git

- [Obsidian Git 플러그인 \(GitHub, Vincent03\)](#)
- [Obsidian Git 시작하기 \(공식 문서\)](#)

Node.js (Claude Code 사전 준비물)

- [Node.js 다운로드 \(공식\)](#)

참고: 위 링크는 작성 시점 기준이며, 도구 버전 업데이트로 화면·명령이 달라질 수 있습니다. 공식 문서의 최신 내용을 우선합니다.